

Tytuł projektu:

Grzmotoptak – Platforma do zarządzania wydarzeniami i rejestracją uczestników.

Opis projektu:

Celem projektu jest stworzenie platformy umożliwiającej przeglądanie, tworzenie i zarządzanie wydarzeniami, takimi jak warsztaty, konferencje, koncerty, imprezy czy spotkania.

Niezałogowani użytkownicy mogą sobie pograć w kółko i krzyżyk na stronie domyślnej oraz przeglądać dostępne wydarzenia.

Użytkownicy zalogowani mogą przeglądać dostępne wydarzenia, tworzyć nowe wydarzenia, usuwać oraz edytować przez siebie dodane. Również mogą zapisywać swoje uczestnictwo w wydarzeniach, które inni użytkownicy stworzyli oraz przeglądać wydarzenia do których się zapisali. Sam twórca wydarzenia jest automatycznie przypisany do własnego wydarzenia i nie może się z niego usunąć. Administratorzy mają możliwość dodawania i edytowania wydarzeń, przeglądania listy użytkowników, dodawania, usuwania oraz zmieniania roli użytkowników.

Autorzy:

Bartosz Cieżuch, Wiktor Stolarski

Specyfikacja wykorzystanych technologii:

specyfikacja wykorzystanych technologii

Język programowania: C#, HTML, CSS, JavaScript

Framework: ASP.NET MVC

Wersja .NET: .NET 8.0

Baza danych: MS SQL Server

Narzędzie ORM: Entity Framework Core, ASP.NET Core Identity

Inne biblioteki/narzędzia: Bootstrap, jQuery,

Środowisko IDE: Visual Studio 2022

Instrukcje pierwszego uruchomienia projektu :

1. Uruchomić visual studio ---> kliknij "klonuj repozytorium", następnie **wklej adres Https** (ten z moodle z prac do oceny) ---->kliknij klonuj.
2. Należy przejść do konsoli menadzer pakietów nugets i tam zrobić **Update Database**
3. Następnie należy uruchomić program
4. Aby zalogować się na admina to należy wpisać (email: admin@example.com oraz hasło: **Admin@123**) lub jesli chcesz stworzyć użytkownika kliknij "zarejestruj się",podaj email i hasło oraz podaj ponownie hasło aby potwierdzić poprawność hasła a następnie naciśnij przycisk "register"
5. Inna metoda tworzenia użytkownika lub nowego administratora: Aby utworzyć nowego użytkownika lub administratora, należy **(koniecznie być zalogowanym na koncie administratora)** jeśli chcemy dodać zwykłego usera/administratora przechodzimy do "Panel Administratora" --->"zarządzaj użytkownikami" w polu "dodaj użytkownika" podajemy email dla użytkownika oraz hasło i wybieramy jedną z dwóch opcji ról(User,Admin)
6. Jeśli w punkcie 5 stworzyliśmy użytkownika/Administratora możemy się teraz zalogować z takimi danymi ,które podaliśmy wcześniej.

Opis struktury projektu:

- **/Controllers:** Zawiera kontrolery aplikacji:
 - `AdminController.cs`: Kontroler do zarządzania użytkownikami, rolami i wydarzeniami.
 - `WydarzeniaController.cs`: Kontroler odpowiedzialny za logikę wydarzeń, w tym CRUD dla wydarzeń.
 - `HomeController.cs`: Kontroler główny, najprawdopodobniej z funkcją wyświetlania strony głównej.
- **/Models:** Zawiera klasy modeli i encje bazy danych:
 - `ApplicationUser.cs`: Rozszerza `IdentityUser`, jeśli chcesz dodać niestandardowe właściwości dla użytkowników.
 - `Zapisy.cs`: Model reprezentujący zapis na wydarzenie.
 - `wydarzenia.cs`: Model reprezentujący wydarzenie.
- **/Views:** Zawiera widoki (Razor):
 - `/Home`: Widok strony głównej (np. `Index.cshtml`).
 - `/Wydarzenia`: Widoki związane z wydarzeniami, np. lista wydarzeń (`Index.cshtml`), formularz edycji (`Edit.cshtml`).
 - `/Admin`: Widoki panelu administratora, np. zarządzanie użytkownikami i wydarzeniami (`Users.cshtml`, `Events.cshtml`, `PanelAdministratora.cshtml`).
- **/Data:** Zawiera kontekst bazy danych i metodę do seeda:
 - `ApplicationDbContext.cs`: Klasa kontekstu bazy danych, zawierająca `DbSet`y dla encji (np. `DbSet<wydarzenia>`, `DbSet<Zapisy>`).
- **/Migrations:** Zawiera pliki migracji bazy danych generowane przez Entity Framework.
- **/wwwroot:** Zawiera zasoby statyczne:
 - `/css`: Pliki CSS, np. `site.css`.
 - `/js`: Skrypty JavaScript.
 - `/images`: Obrazy, np. logo aplikacji lub zdjęcia wydarzeń.
- **/Services:** Zawiera klasy logiczne, które są używane przez kontrolery:
 - `EventService.cs`: Usługa odpowiedzialna za logikę związaną z wydarzeniami.

- `UserService.cs`: Usługa odpowiedzialna za logikę związaną z użytkownikami, np. tworzenie nowych użytkowników lub aktualizacja ról.
 - **/Areas**: Dodatkowe obszary w aplikacji, np. obszar inicjalizacyjny użytkownika(`Identity`), w którym mogą być oddzielne kontrolery i widoki:
 - `Register.cshtml`: Widok rejestracji użytkownika wraz z logiką działania
-

Wylistowane wszystkie modele:

1. Model: Zapisy

Reprezentuje zapis użytkownika na wydarzenie.

Pola:

1. Id (int)

- Identyfikator rekordu.
- **Opis**: Klucz główny modelu.
- **Walidacja**: Brak.

2. UserId (string)

Identyfikator użytkownika (`IdentityUser`).

Opis: Klucz obcy odnoszący się do użytkownika z systemu tożsamości (`IdentityUser`).

Walidacja: Brak.

3. EventId (int)

Identyfikator wydarzenia.

Opis: Klucz obcy odnoszący się do modelu wydarzenia.

Walidacja: Pole obowiązkowe ([Required]).

4. User (IdentityUser)

Powiązanie z użytkownikiem systemu tożsamości.

Opis: Nawigacyjne pole w relacji do tabeli użytkowników.

Walidacja: Brak.

5. Event (wydarzenia)

-Powiązanie z wydarzeniem.

Opis: Nawigacyjne pole w relacji do modelu wydarzenia.

Atrybut: [ForeignKey("EventId")].

Walidacja: Brak.

Konstruktor:

6. Zapisy(string userId, int eventId)

Opis: Umożliwia inicjalizację pól UserId i EventId.

Model: wydarzenia

Reprezentuje pojedyncze wydarzenie.

Pola:

1. Id (int)

- Identyfikator wydarzenia.
- **Opis:** Klucz główny modelu.
- **Walidacja:** Pole obowiązkowe ([Required]).

2. Tytuł (string)

Tytuł wydarzenia.

Opis: Krótkie określenie tematu wydarzenia.

Walidacja: Pole obowiązkowe ([Required]).

3. Opis (string)

Szczegółowy opis wydarzenia.

Opis: Zawiera informacje o charakterze i treści wydarzenia.

Walidacja: Pole obowiązkowe ([Required]).

4. Data (DateTime)

Opis: Data wydarzenia.

Opis: Informacja o dniu i godzinie rozpoczęcia.

Walidacja: Pole obowiązkowe ([Required]).

5. Typ (string)

Typ wydarzenia.

Opis: Określa kategorię wydarzenia, np. Konferencja, Warsztat.

Walidacja: Pole obowiązkowe ([Required]).

6. tworca_id (string?)

Identyfikator użytkownika-twórcy.

Opis: Klucz obcy odnoszący się do tabeli użytkowników (IdentityUser).

Walidacja: Pole opcjonalne.

Atrybut: [ForeignKey("tworca_id")].

7. User (IdentityUser?)

Powiązanie z twórcą wydarzenia.

Opis: Nawigacyjne pole w relacji do użytkownika systemu tożsamości.

Walidacja: Brak.

8. Obrazek (string?)

Ścieżka lub URL do obrazka.

Opis: Opcjonalny element graficzny związany z wydarzeniem.

Walidacja: Brak.

Metody:

- **SetTworcald(string userId)**

Ustawia identyfikator twórcy, jeśli pole tworca_id jest puste.

Opis: Zapobiega nadpisaniu istniejącego tworca_id.

Wylistowane kontrolery wraz z metodami:

Kontroler: wydarzeniaController

Metody:

1. Index

- Metoda HTTP:** GET
- Parametry:** Brak
- Opis działania:** Wyświetla listę wszystkich wydarzeń.
Pobiera identyfikator aktualnie zalogowanego użytkownika i identyfikatory wydarzeń, na które jest zapisany. Wydarzenia są przekazywane do widoku.
- Zwracane dane:** Widok z listą wydarzeń.

2. Details

- Metoda HTTP:** GET
- Parametry:** int? id (opcjonalny)
- Opis działania:** Wyświetla szczegóły wybranego wydarzenia na podstawie podanego ID. Jeśli ID nie istnieje, zwraca błąd 404.

d. **Zwracane dane:** Widok ze szczegółami wydarzenia.

3. Create

a. **Metoda HTTP:** GET, POST

b. **Parametry (GET):** Brak

c. **Parametry (POST):** wydarzenia (model wydarzenia),
IFormFile obrazek (opcjonalny obrazek wydarzenia)

d. **Opis działania:**

- i. GET: Wyświetla formularz tworzenia nowego wydarzenia.
- ii. POST: Dodaje nowe wydarzenie do bazy danych.
Obsługuje przesyłanie obrazu, zapisując jego ścieżkę.

e. **Zwracane dane:**

- i. GET: Widok formularza.
- ii. POST: Przekierowanie do widoku Index lub ponowne wyświetlenie formularza w przypadku błędu.

4. Edit

a. **Metoda HTTP:** GET, POST

b. **Parametry (GET):** int? id (opcjonalny)

c. **Parametry (POST):** int id, wydarzenia (model wydarzenia),
IFormFile obrazek (opcjonalny obrazek wydarzenia)

d. **Opis działania:**

- i. GET: Wyświetla formularz edycji wybranego wydarzenia. Sprawdza, czy użytkownik ma odpowiednie uprawnienia.
- ii. POST: Aktualizuje dane wydarzenia w bazie.
Obsługuje zmianę obrazu, jeśli został przesłany nowy.

e. **Zwracane dane:**

- i. GET: Widok formularza.
- ii. POST: Przekierowanie do widoku Index lub ponowne wyświetlenie formularza w przypadku błędu.

5. Delete

a. **Metoda HTTP:** GET, POST

b. **Parametry (GET):** int? id (opcjonalny)

c. **Parametry (POST):** int id

d. **Opis działania:**

- i. GET: Wyświetla formularz potwierdzenia usunięcia wydarzenia.
- ii. POST: Usuwa wydarzenie na podstawie ID, jeśli użytkownik ma odpowiednie uprawnienia.

e. Zwracane dane:

- i. GET: Widok potwierdzenia.
- ii. POST: Przekierowanie do widoku Index.

6. MojeWydarzenia

- a. **Metoda HTTP:** GET
- b. **Parametry:** Brak
- c. **Opis działania:** Wyświetla listę wydarzeń, na które zalogowany użytkownik jest zapisany.
- d. **Zwracane dane:** Widok z listą zapisanych wydarzeń.

7. Przypisz

- a. **Metoda HTTP:** POST
- b. **Parametry:** int eventId
- c. **Opis działania:** Zapisuje zalogowanego użytkownika na wybrane wydarzenie, jeśli nie jest jeszcze zapisany.
- d. **Zwracane dane:** Status (OK lub błąd).

8. Wypisz

- a. **Metoda HTTP:** POST
- b. **Parametry:** int eventId
- c. **Opis działania:** Usuwa zapis użytkownika na wybrane wydarzenie.
- d. **Zwracane dane:** Przekierowanie do widoku MojeWydarzenia.

Kontroler: AdminController

Metody:

1. Index

- a. **Metoda HTTP:** GET
- b. **Parametry:** Brak
- c. **Opis działania:** Wyświetla stronę główną panelu administratora.

d. **Zwracane dane:** Widok panelu administratora.

2. Użytkownicy

a. **Metoda HTTP:** GET

b. **Parametry:** Brak

c. **Opis działania:** Wyświetla listę wszystkich użytkowników z ich rolami.

d. **Zwracane dane:** Widok z listą użytkowników.

3. ChangeRole

a. **Metoda HTTP:** POST

b. **Parametry:** string userId, string role

c. **Opis działania:** Zmienia rolę użytkownika. Najpierw usuwa wszystkie dotychczasowe role użytkownika, a następnie przypisuje nową. Nie pozwala zmienić roli użytkownika głównego administratora (SuperAdminEmail).

d. **Zwracane dane:** W przypadku powodzenia: Przekierowanie do widoku „Użytkownicy”. W przypadku błędu: Komunikat błędu.

4. CreateUser

a. **Metoda HTTP:** POST

b. **Parametry:** string email, string password, string role

c. **Opis działania:** Tworzy nowego użytkownika, przypisuje mu podaną rolę i zapisuje go w bazie danych. Jeśli rola nie istnieje, zwraca błąd.

d. **Zwracane dane:** W przypadku powodzenia: Przekierowanie do widoku „Użytkownicy”. W przypadku błędu: Widok „Użytkownicy” z odpowiednimi komunikatami walidacyjnymi.

5. GetUsersWithRoles

a. **Metoda HTTP:** Wywoływana wewnętrznie (helper).

b. **Parametry:** Brak

c. **Opis działania:** Pobiera listę użytkowników wraz z przypisanymi rolami. Metoda pomocnicza wykorzystywana przez inne metodę, CreateUser.

d. **Zwracane dane:** Lista użytkowników i ich ról w formacie (IdentityUser User, IList<string> Roles).

6. DeleteUser

- a. **Metoda HTTP:** POST
- b. **Parametry:** string id
- c. **Opis działania:** Usuwa użytkownika na podstawie jego ID.
Nie pozwala usunąć super administratora (SuperAdminEmail).
- d. **Zwracane dane:** W przypadku powodzenia: Przekierowanie do widoku „Użytkownicy” z komunikatem sukcesu. W przypadku błędu: Przekierowanie do widoku „Użytkownicy” z komunikatem błędu.

7. EventsAsync

- a. **Metoda HTTP:** GET
- b. **Parametry:** Brak
- c. **Opis działania:** Wyświetla listę wszystkich wydarzeń w systemie.
- d. **Zwracane dane:** Widok z listą wydarzeń.

opis systemu użytkowników:

System wspiera różne role użytkowników:

1. Gość:

- a. Ma dostęp jedynie do publicznej części serwisu (Strona główna, podgląd listy wydarzeń).
- b. Nie może tworzyć, edytować ani zapisywać się na wydarzenia.
- c. Może pograć w kóło i krzyżyk na stronie głównej(Home)

2. Zalogowany Użytkownik:

- a. Może zapisywać się na wydarzenia oraz przeglądać swoje zapisy.
- b. Może tworzyć wydarzenia i zarządzać swoimi wydarzeniami (edycja, usuwanie).

3. Administrator:

- a. Ma pełny dostęp do systemu.
- b. Ma możliwość dodawania, usuwania, edytowania wydarzeń.
- c. Może zarządzać wszystkimi wydarzeniami i użytkownikami.

Nadawanie Ról

- Role są przypisywane użytkownikom na podstawie ich konta w systemie tożsamościowym (Identity).
- Administrator może zmienić rolę użytkownika w panelu zarządzania użytkownikami.

Różnice między użytkownikami zalogowanymi a gośćmi

- **Goście:**
 - Mogą jedynie przeglądać dostępne wydarzenia bez interakcji.
 - Nie mają dostępu do szczegółowych funkcji systemu.
- **Zalogowani użytkownicy:**
 - Mogą tworzyć wydarzenia, zapisywać się na nie oraz przeglądać swoje zapisy.
 - Widzą dodatkowe informacje, takie jak status zapisów czy szczegóły wydarzeń, na które się zapisali.

Powiązane informacje:

- **Z użytkownikiem:**
 - Identyfikator użytkownika (UserId).
 - Lista zapisów na wydarzenia (Zapisy).
 - Stworzone przez użytkownika wydarzenia (events).

Krótką charakterystyka najciekawszych funkcjonalności :

1. Gra kółko i krzyżyk

- a. Gra kółko i krzyżyk na stronie głównej

2.Zarządzanie zapisami na wydarzenia:

- a. Użytkownik może zapisać się lub wypisać z wybranego wydarzenia w prosty sposób.
- b. System uniemożliwia, zapisanie się na to samo wydarzenie wielokrotnie.
- c. System podlicza ilość użytkowników zapisanych do udziału w danym wydarzeniu i wyświetla je w sekcji przypisanych wydarzeń twórcy wydarzenia.

3.Przypisywanie twórców do wydarzeń:

- a. Każde wydarzenie ma przypisanego twórcę. System weryfikuje uprawnienia użytkownika przy próbie edycji lub usunięcia wydarzenia.
- b. Twórca wydarzenia automatycznie przypisywany jest jako uczestnik wydarzenia które stworzył.

4.Zarządzanie użytkownikami jako Administrator:

- a. Administrator może dodać użytkowników bezpośrednio w panelu Admina
- b. Administrator może zmieniać role innym użytkownikom (Admin, User)